txt2tex Relational Databases Reference

Relational databases: primary keys, relational algebra, bindings, GROUP/UNGROUP.

https://github.com/jmf-pobox/txt2tex

Document Structure

`=== Title ===`	`\section *{Title}`
`* Solution N **`	Bold heading with spacing
`(a) Text`	Part label
`TEXT: paragraph`	Prose block (smart formula detection)
`LATEX: \cmd`	Raw LaTeX passthrough
`PAGEBREAK:`	Page break
`LINEBREAK:`	Vertical gap (`\medskip`) — between blocks, no page break

`TITLE: ...`	Document title
`AUTHOR: ...`	Author name
`DATE: ...`	Date string
`BIBLIOGRAPHY: f.bib`	BibTeX file
`[cite key]`	`\citep {key}`

Logic

`p land q`	$p \land q$
`p lor q`	$p \lor q$
`lnot p`	$\neg\, p$
`p => q`	$p \Rightarrow q$
`p <=> q`	$p \Leftrightarrow q$
`forall x : S | P`	$\forall\, x : S \bullet P$
`exists x : S | P`	$\exists\, x : S \bullet P$
`exists_1 x : S | P`	$\exists_1\, x : S \bullet P$
`lambda x : S | E`	$\lambda\, x : S \bullet E$
`mu x : S | P`	$\mu\, x : S \bullet P$

Note: Use `land`, `lor`, `lnot`. English `and`/`or`/`not` are not supported.

Sets & Types

`x elem S`	$x \in S$
`x notin S`	$x \notin S$
`A subset B`	$A \subseteq B$
`A union B`	$A \cup B$
`A inter B`	$A \cap B$
`A setminus B`	$A \setminus B$
`power S`	$\mathbb{P}\, S$
`F S`	$\mathbb{F}\, S$
`{x : S | P}`	Set comprehension
`A cross B`	$A \times B$
`N / Z / N1`	$\mathbb{N}$ / $\mathbb{Z}$ / $\mathbb{N}_1$
`n..m`	$n \mathinner{..} m$

Z Notation Blocks

`given A, B`	Given types: $[A, B]$
`T ::= a | b`	Free type
`name == expr`	Abbreviation

`axdef` / `schema Name` / `gendef [X]`

`declarations`	Variable declarations
`where`	Separator
`predicates`	Constraining predicates
`end`	Close the block

Usage

`txt2tex f.txt`	→ f.tex + f.pdf
`txt2tex f.txt --tex-only`	→ f.tex only
`txt2tex -i`	Interactive REPL
`txt2tex --check-env`	Verify dependencies

txt2tex Relational Databases — By Example

Primary keys, relational algebra, bindings, GROUP/UNGROUP.

Primary Keys

Mark a primary-key attribute with `pk` in a named schema body. Fuzz-compatible.

You write:

```
schema Class
  pk class : ClassName
  country  : CountryName
  bore     : N
end
```

You get:

Class

$class : ClassName$
$country : CountryName$
$bore : \mathbb{N}$

$$\mathrm{PK}(\mathrm{Class}) = \{class\}$$

Composite key: two `pk` lines $\Rightarrow \mathrm{PK}(S) = \{a, b\}$. Comma-separated names on one `pk` line also work. `pk` is only valid inside a named `schema`; not allowed in `axdef`, `gendef`, or inline schema text.

Relational Algebra

`sigma[p](R)`	$\mathrm{Restrict}_p(R)$ — restrict
`pi[A,B](R)`	$\mathrm{Project}\{A, B\}(R)$ — project
`rho[A as B](R)`	$\mathrm{Rename}_{A \to B}(R)$ — rename
`R bowtie S`	$R \otimes S$ — natural join
`R bowtie [p] S`	$\mathrm{Join}_p(R, S)$ — theta-join
`R div S`	$R \mathbin{\mathsf{div}} S$ — division

Naming a query. Use `Name == expr` for any right-hand side — pure Z or relational algebra.

Fuzz-compatible: write algebra as free-standing lines in the document, not inside `zed/axdef/schema` blocks. Schemas and axdefs in the same document still type-check.

Long expressions: break at any operator — bare newline or trailing \. Break *before* `setminus`, not after.

You write:

```
BigGuns == pi[class,country](
  sigma[bore >= 16](Class))
```

You get:

$$BigGuns \mathrel{\widehat=} \mathrm{Project}\{class, country\}(\mathrm{Restrict}_{bore \geq 16}(Class))$$

Z Bindings & Set Comprehension

Z RM §3.7 binding brackets construct labelled tuples.

`{| name == e |}`	$\langle name == e \rangle$
`{| a == e, b == f |}`	multiple components
`{| |}`	empty binding

Multi-typed comprehension with binding:

You write:

```
{ s : Ship; c : Class |
  s.class = c.class .
  {| name == s.name |} }
```

You get:

$$\{s : Ship;\ c : Class \mid s.class = c.class \bullet \langle name == s.name \rangle\}$$

Use `;` to separate variable-type pairs inside `{ }`. Binding components are **comma**-separated (Z RM §3.7).

Fuzz-compatible: write bindings as free-standing expressions in the document, not inside `zed/axdef/schema` blocks.

GROUP & UNGROUP

Date's nested-relation operators.

`R group ({A,B} as alias)`	R GROUP$(\{A, B\}$ AS $alias)$
`R ungroup alias`	R UNGROUP $alias$

You write:

```
Members group ({name,email} as contact)
Members ungroup contact
```

You get:

$$Members \text{ GROUP } (\{name, email\} \text{ AS } contact)$$
$$Members \text{ UNGROUP } contact$$

Fuzz-compatible: write GROUP/UNGROUP as free-standing expressions in the document, not inside `zed/axdef/schema` blocks.

Quick Reference

`pk a :  T`	primary key in schema
`pk a, b :  T`	composite pk
`sigma[p](R)`	restrict
`pi[A](R)`	project
`rho[A as B](R)`	rename attr
`R bowtie S`	natural join ($\otimes$)
`R div S`	division
`{| a == e |}`	binding bracket
`{S; C | P . E}`	multi-typed set comp
`R group ({A} as x)`	group attrs
`R ungroup x`	ungroup attr

Identifier Decoration

`count'`	$count'$ (after-state)
`in?`	$in?$ (input)
`out!`	$out!$ (output)
`x?'`	$x?'$ (runs may combine)
`'sunk'`	'sunk' (string literal)

Decoration attaches to any identifier. Reserved words (`theta`, `defs`, `hide`, `project`, `Delta`, `Xi`) cannot be decorated.